

Here is the framework and template for **Stage 7: AI vs me**, designed to structure your comparative code review after you generate the AI code in quarantine.

Setting Up the Isolation Zone

To make sure your hand-crafted, production-ready code remains pristine, set up your workspace directories like this before executing the rematch:

bash

```
my-auth-gateway/
├── .env
├── .gitignore
├── README.md           # Your original project guide
├── stage1-6-code/      # Your trusted hand-written implementation
│   ├── main.js (or main.py)
└── ai-version/         # The Quarantine Zone
    ├── main.js (or main.py)
```

Use code with caution.

Step 1: Draft Your Engineering Prompt

Do not copy-paste previous steps! Try writing your prompt from scratch into the AI generator window. Here is a strong target specification checklist you should aim to describe purely from memory:

1. **The Architecture:** Specify Node/Express or Python/FastAPI using the Supabase client library.
2. **The Endpoints & Strict Status Codes:** Enforce `POST /auth/signup` (201), `POST /auth/login` (200), `GET /public/info` (200), `POST /auth/logout` (204), and `GET /protected/profile` (200).
3. **The Validation Barriers:** Explicitly demand `400 Bad Request` for missing client inputs and `401 Unauthorized` for missing/invalid bearer tokens.
4. **The Centralized Guard:** Mandate that protected routes must be secured using a singular, automated middleware/dependency layer that extracts the token and validates it via `getUser()`.

5. OpenAPI Documentation: Request interactive Swagger/OpenAPI setup with a mounted global `HTTP Bearer` security scheme definition.

Step 2: The Audit Checklist (Code Review)

Once the AI prints its script, paste it into your `ai-version/` directory and run the comparative diff tool in your terminal:

```
bash
```

```
git diff --no-index stage1-6-code/main.js ai-version/main.js
```

Use code with caution.

When reviewing the generated code, carefully evaluate it against these critical areas:

- **Token Extraction Stability:** Check how it handles the `Authorization` header. Does it use rigid array splitting (`split(' ')[1]`) that will throw an uncaught runtime error and crash the entire server process if a client sends an empty `Authorization: Bearer` string?
- **The Supabase Trust Trap:** Did it correctly evaluate the network error object returned by the Supabase SDK? Look closely at its conditional statements:

```
javascript
```

```
// 🚨 SECURITY FLAW: Trusting data existence instead of checking the  
const { data } = await supabase.auth.getUser(token);  
if (data) { ... }
```

Use code with caution.

If an expired token is sent, Supabase might still return a data object stub containing an internal error payload. If the AI didn't explicitly check `if (error || !data?.user)`, it may have accidentally let forged users pass right through the door.

- **Leaky Configurations:** Check if it tried to initialize the client with `process.env.SUPABASE_SERVICE_ROLE_KEY` instead of the `ANON_KEY`. Using the service role key bypasses Row Level Security entirely, turning your client into a superuser.

Step 3: Update Your Main `README.md`

Append this exact evaluative block to your repository documentation to clear the checkpoint:

markdown

```
## 🏆 AI vs Me – Engineering Rematch Analysis

### 💬 Original Prompt Used
"Create a [NodeJS Express / Python FastAPI] REST API backed by Supabase A

### ⚖️ Architectural Differences & Anomalies

1. **Token Processing Resilience:**
  * **My Approach:** Safely parsed headers with boundary checks, validat
  * **AI Approach:** [e.g., The AI used a fragile split statement that t

2. **Security & Exception Handling:**
  * **My Approach:** Explicitly captured the `error` block returned by t
  * **AI Approach:** [e.g., The AI only checked `if (!user)` inside a tr

3. **Silent Assumptions (What I Forgot to Specify):**
  * **The Lesson:** My prompt did not specify how to handle global excep

### 🔄 The Prompt Evolution
*Regeneration Sentence:* After adding `Ensure all error responses strict
```

Use code with caution.

Deploy the Rematch Checkpoint

Stage the quarantine folder and your updated notes, then lock down Stage 7 in your git timeline:

bash

```
git add ai-version/ README.md
git commit -m "Stage 7: AI vs me (AI code stays in its own folder/branch)"
git push origin main
```

Use code with caution.

You have now completed the entire authentication microservice lifecycle from scratch! What would you like to build next?

- Refactor the project to run on completely serverless **Edge Functions**
- Add **Unit Testing** with Jest/PyTest to mock the Supabase auth responses
- Connect this backend to a **Frontend framework (React/Vue)** configuration

